A FIELD REPORT FROM A WORKING SYSTEM · ALGORITHMIC DISCOVERY · HONEST EVALUATION

The Recursive Discovery Engine

Building a machine that cannot fool itself — and what it revealed about discovery, verification, and the limits of AI capability.

A small, runnable system pits three AI roles against one another — a Builder that proposes, an Adversary that disproves with executable evidence, and a Synthesizer that alone may declare victory — over twelve generations of escalating rigor. Its most important result is not an algorithm it discovered. It is what happened *every time the measurement was made more honest*: the prior discoveries evaporated. This is a report on that process, written for readers tracking the frontier of AI, where the hard problem is no longer generating a clever candidate but *trusting your own measurement of whether the candidate is real.*

Subject: the Recursive Discovery Engine (RDE), v0 → v11 · **Codebase:** Python, standard-library core, OpenRouter LLM transport · **Scope:** top-K selection, streaming heavy-hitters, cache eviction

Audience: people interested in the frontier of AI · **Stance:** the same scrupulous honesty the system enforces on itself

00 Abstract & contents

The Recursive Discovery Engine is an answer, in miniature, to a question that sits at the center of advanced AI: **when a system claims it discovered something, how do you know it actually did — rather than found a way to look like it did?** RDE's answer is architectural. It makes the score *verifiable by construction* (the thing being graded literally cannot write to its own grade), it applies adversarial pressure from a *different* model than the one proposing, it hands the only judge that may terminate a set of facts it is forbidden to rationalize away, and it adopts one cultural rule above all: *a result that lowers the headline is a success.*

Over a dozen versions, that discipline was turned on the project's own claims. Version 7 reported three machine-discovered cache policies that "beat the human baseline." Version 8 made the metric unfakeable and the honest count fell to one. Version 9 added adaptive primitives and earned the project's first genuinely *invented* wins. Version 10 raised the bar to the humans' own adaptive algorithms (ARC, LeCaR) and, surprisingly, the wins held. Version 11 broadened that bar to the strongest known policies, added multi-seed measurement, swept the cost accounting, and drew a fresh hold-out family of workloads — and the streak **comprehensively ended**. Zero of ten prior winners survived; the two that still looked good on the home benchmark lost decisively on freshly-drawn cousins of the same problems and were flagged as overfit. That null result, reported plainly, is the most trustworthy thing the project has produced — and the clearest lesson it offers the field.

01 Why this belongs at the frontier
The evaluation crisis, reward hacking, and the difference between discovery and interpolation.

02 The machine
Three roles, the core loop, the Empirical Grounding Rule, and the anti-rationalization design.

03 The problem it works on
From top-K to heavy-hitters to cache eviction; why each move; the competitive-ratio metric.

04 The central invention: a number the generator cannot write
The trusted driver, the cheat test, and verification-by-construction.

05 The ladder of honesty
The version-by-version arc in which each gain in rigor destroyed a prior headline.

06 v11: the design constitution and the end of the streak
Unfakeable / alignable / insulatable; the leak rule; the rubric scrub; the hold-out finding.

07 What this teaches about the frontier
Seven transferable lessons for anyone evaluating capable models.

08 Limits, open questions, and what comes next
The pending "capability without idiom" run, generalization, and honest caveats.

Roles	Builder · Adversary · Synthesizer	One vendor per role	shared-blind-spot defense
Core dependencies	Python stdlib only	Test suite	169 tests
Default models	gpt-5.1-codex · gemini-3.1-pro · opus-4.8	Quality metric	hit_rate / Belady-OPT
v11 robust wins	0 of 10 · 0 of 14	Governing rule	lowering the headline = success

01 Why this belongs at the frontier of AI

For most of the field's history the binding constraint was generation: could a model produce a candidate solution worth looking at? That constraint is dissolving. Capable models now propose fluent, plausible, often-correct artifacts faster than anyone can check them. The constraint has moved. The frontier question today is **verification**: can we measure whether a capable system actually did the thing — reliably, repeatably, and in a way the system itself cannot quietly subvert?

This is not an academic worry. It is the same problem under three names that the field cares about most:

- **Reward hacking.** A system optimized against a proxy will exploit the gap between the proxy and the goal. The more capable the optimizer, the more reliably it finds that gap. An evaluation you trusted at one capability level can quietly become a target at the next.
- **The evaluation crisis.** Benchmarks saturate, leak, and get gamed. A number that once meant "good" comes to mean "good at the benchmark." Distinguishing real capability from benchmark-fit is now a first-order scientific problem, not a footnote.
- **Discovery vs. interpolation.** When a model produces something that scores well, did it *invent* a mechanism or *recombine* one it was shown? The distinction is the whole ballgame for AI-for-science, and it is notoriously slippery to define, let alone measure.

The Recursive Discovery Engine is a working laboratory for exactly these questions, shrunk to a size where every claim can be checked by hand. Its domains — selecting the top-K elements of an array, tracking frequent items in a stream, choosing what to evict from a cache — are deliberately humble. They have *known optima* or strong, well-studied human baselines, which means the question "did the machine actually beat the best human approach?" has a real, computable answer rather than a vibe. That is what makes RDE valuable as a frontier artifact: it is small enough to be honest about, and structured exactly like the large problem.

> **THE THESIS IN ONE LINE**
>
> The interesting output of RDE is not a discovered algorithm. It is a demonstration that a discovery system can be built so that *its own success metric resists being gamed by the intelligence operating inside it* — and a candid record of what such a system finds when you stop letting it flatter itself.

Read that way, the rest of this report is a case study in a single discipline: how to wire an AI system so that capability is pointed at the problem and cannot leak into the scorekeeping — and what is left standing, as a genuine result, once it cannot.

02 The machine

RDE runs a loop. A **Builder** proposes approaches. Each proposal is executed and graded by a trusted harness. An **Adversary** then tries to *break* the survivors with hostile inputs — and is believed only when it produces an input that actually breaks them. A **Synthesizer** maintains the high-level view and is the *only* role permitted to declare convergence or stop. The loop repeats until the Synthesizer accepts a winner or the budget runs out.

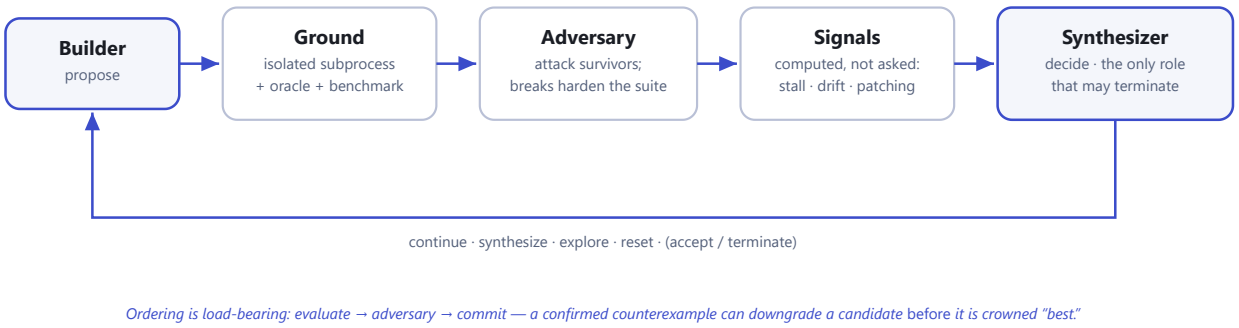


Figure 1 — The core loop in `engine.py`. The same loop drives both the zero-cost deterministic reference roles and the live LLM roles; only the trio plugged in differs.

Three roles, three different model vendors

Each role has two implementations: a *deterministic* reference (zero API cost, what the test suite exercises, and the fallback when a model fails) and an *LLM* flavor. The most important design choice in the LLM configuration is that the three roles are assigned to three **different model families**.

> **THE SHARED-BLIND-SPOT DEFENSE**
>
> If the Builder, Adversary, and Synthesizer all ran the same base model, they would share systematic errors — and the Adversary could not catch what the Builder cannot see. Assigning a distinct vendor to each role is the cheapest structural defense against correlated failure. In a live run, the `claude-haiku` Synthesizer correctly tagged the `gpt-5-mini` Builder's full-sort proposal as "recombination of known techniques" — outside-perspective friction a single model cannot give itself.

The Empirical Grounding Rule

Nothing advances on an assertion. Every candidate is run in an isolated subprocess with a hard timeout, checked against a brute-force **oracle**, benchmarked, and memory-profiled. Correctness is a *hard gate*: a wrong approach is disqualified, never merely down-weighted. The rule that gives the whole system its spine is simple and absolute:

> **No field of the evidence record is ever set by a model's claim. Every number is produced by trusted code that ran the candidate.**

The Adversary obeys the same rule. It does not get to *say* a candidate is fragile; it proposes an input, the harness runs that input against the oracle, and the harness decides what broke. A confirmed counterexample does two things at once: it downgrades the candidate *and* is folded permanently into the trusted evaluation suite. The system's eyes grow over the course of a run.

The anti-rationalization design — the key idea

This is the part most worth carrying to larger systems. **The Synthesizer does not get to decide whether the process is drifting.** A separate module computes objective signals from history — has fitness stalled over a window? are the last several proposals cosmetic rewrites of one idea? is a failure class recurring? — and hands those facts to the Synthesizer as inputs it cannot override. The Synthesizer's decision is then checked against hard guardrails: it *cannot* accept with no correct approach in hand, and it *cannot* continue through confirmed local patching.

> **WHY THIS MATTERS BEYOND RDE**
>
> The failure mode it prevents is the one every operator of capable models has seen: the model cheerfully reports "great progress!" while emitting the third near-identical variant of a stalled idea. You do not fix that by asking the model to be more honest. You fix it by *computing* the thing you would otherwise have asked about, and making the model's freedom conditional on the computed facts. Judgment the model could rationalize is moved out of the model.

Bounded memory and a security perimeter

Two supporting details matter for credibility. First, the archive handed to the LLM roles is capped to the Pareto frontier plus a little context, so a fifty-iteration run prompts with as much state as a five-iteration run — the system cannot "win" by drowning its judges in history. Second, because the harness executes model-written code, it defends in layers: a separate stripped Python process per candidate, an AST static screen that allow-lists imports and forbids code-execution and introspection builtins (the `eval`, `exec`, `compile`, `getattr`, `open` family) and rejects dunder access outright, a regex denylist, and an RSS watchdog. The report is candid that this is adequate for a trusted local research setting, not a hardened sandbox — an honesty about its own boundaries that recurs everywhere in the project.

03 The problem it works on

RDE changed domains twice, and each move was forced by an honest limitation of the one before. The progression is itself instructive: it is the story of choosing a measuring instrument honest enough to register the thing you are trying to detect.

Top-K (v0–v1): a closed problem

The first domain was selecting the K largest elements of an array. It worked — the engine explored several strategies, disqualified a broken one by execution, fell into the empirical trap of preferring quickselect on paper before measurement showed the heap won, and synthesized an adaptive hybrid that dispatches on K and beats every single strategy. But top-K has a *known optimum* and a pass/fail oracle. Recombination was sufficient; the synthesis win was noise-sized. A closed problem cannot show you the difference between search and discovery.

Heavy-hitters (v2–v6): open, but saturating

So the domain opened: track the most frequent items in a stream under a memory budget, scored by recall against an exact counter, with peak memory as the binding resource. Now quality was continuous and a Pareto frontier of (recall × memory) gave the engine something real to advance. This is where the project earned its first genuine machine-proposed primitive that *beat* the best human op — a threshold-prune rule called `declow`, confirmed real by brute-forcing all 480 human op-combinations. But the win, honestly assessed, lived only on the memory axis; on recall the domain was human-saturated. You cannot demonstrate that a machine out-designs humans on a problem humans already solve optimally.

Cache eviction (v7–v11): real headroom, a true oracle, no dominant human

The final domain is cache replacement: given a trace of accesses and a fixed capacity, decide what to evict. It has three properties that make it the right instrument. There is a true optimum — **Bélády's algorithm**, the clairvoyant policy that evicts whatever is used furthest in the future, verified optimal by brute force on thousands of traces. There is genuine headroom: real human policies sit well below the optimum on adversarial traces. And crucially, *no single human policy dominates* — LRU wins on reuse, LFU on skew, MRU on loops. A win here is unambiguous rather than a technicality.

> **COMPETITIVE RATIO: MAKING REGIMES COMPARABLE**
>
> From v9 on, quality is not raw hit-rate but the **competitive ratio**, `hit_rate / Belady-OPT` per trace — "how close to optimal *here*." Raw hit-rates are dominated by easy regimes; the ratio puts a loop and a skew workload on the same scale, so a cross-regime average actually means something. The headline grade is the weighted competitive-ratio mean over a declared seven-regime benchmark (skew, recency, drift, mixed, plus an adversarial tail of loop, scan, and antagonistic traces). Robustness across regimes is what scores — not a spike on one.

The cost axis is the exact declared capacity (a clean integer), not a noisy memory measurement — a detail that turns out to matter enormously, because it is precisely what later exposed an inflated headline. With a true oracle, a continuous and regime-fair quality metric, and an exact cost axis, the instrument was finally honest enough to register what the machine was — and was not — doing.

04 The central invention: a number the generator cannot write

If this report has one engineering idea to carry away, it is this one. It is the hinge between versions 7 and 8, and it is the concrete, transferable answer to “how do you keep a capable generator from grading its own work.”

The hole in v7

In v7 a candidate policy returned its own score: it computed a hit-rate and reported it, and the verdict trusted that number (rejecting only the physically impossible — a rate above the optimum). The problem is subtle and general. A free-form candidate *sees the whole problem*. Nothing stopped it from computing Bélády’s optimum itself and simply *reporting the optimum* as though it had achieved it. The metric had a channel from the candidate’s intelligence straight into the scoreboard. That channel is the essence of reward hacking.

The fix: separate the logic from the number

Version 8 closed it by construction. A cache candidate no longer returns a score. It provides only *eviction logic* — a function that ranks a key for eviction, plus a memory knob. A **domain-owned trusted driver** — code the engine controls, not the candidate — runs the actual cache simulation, owns the hit counter, and computes the number. The candidate’s intelligence is pointed entirely at *how to evict well*; it has no path to the scoreboard at all.

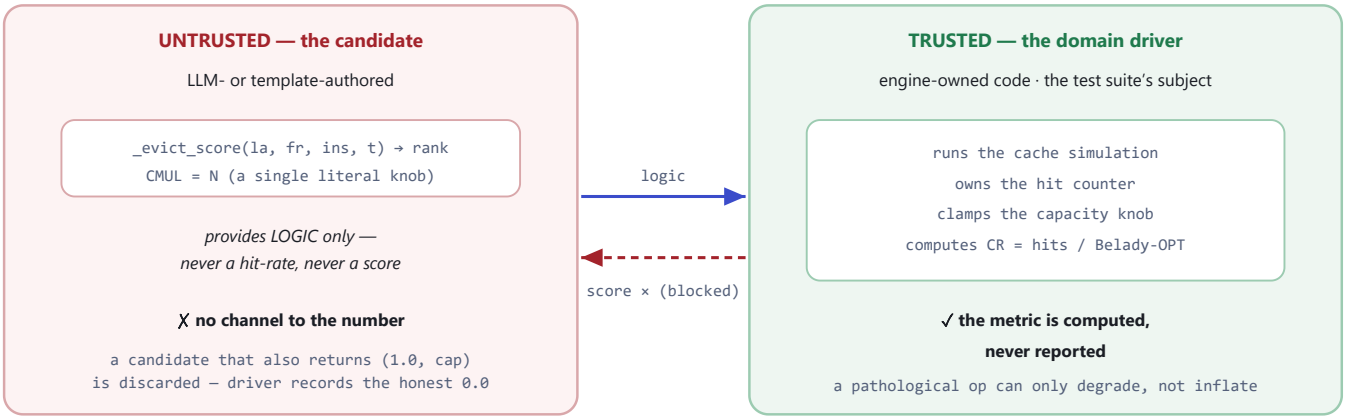


Figure 2 — Verification by construction. The candidate supplies eviction *logic*; trusted, engine-owned code runs the simulation and produces the score. The reward-hacking channel is not policed — it is structurally absent.

The cheat test

The guarantee is pinned by a test that tries to cheat. A candidate is written that also defines a function returning a perfect score. The harness binds the score to the *driver’s* output and discards the candidate’s — so the recorded result is the driver’s honest (poor) hit-rate on a loop, not the perfect score the candidate tried to hand back. “Unfakeable” here is not an adjective; it is a passing test.

THE PATTERN, STATED GENERALLY

Do not ask the system under evaluation for the number. Have it supply only the *mechanism*, and let trusted code you control produce the measurement by running that mechanism. Then make the cost axis exact and the test suite fixed, so two candidates are truly comparable. This is the difference between an evaluation that *asks* and one that *checks* — and it scales to any setting where you can run the artifact instead of reading its self-report.

The design itself was chosen adversarially: three candidate architectures were scored by a panel of judges, and the trusted-driver design won with a perfect soundness score and zero residual cheats found. The same rigor was then turned on v8's own first results — with consequences described next.

05 The ladder of honesty

The spine of the project is a sequence in which **each version's honest self-assessment set the next version's agenda**, and improving the measurement repeatedly destroyed a prior headline. This is the opposite of how benchmark numbers usually move. It is worth seeing the whole arc at once.

Table 1 — Twelve generations. The right-hand column is the honest finding that drove the next move.

VER.	WHAT IT ADDED	THE HONEST FINDING (AND THE AGENDA IT SET)
`v0–v1`	Structured search over top-K; adaptive hybrid synthesis; conceptual memory; computed drift signals.	Top-K is closed (known optimum). Recombination suffices; the synthesis win is noise-sized. *Need an open problem with real headroom.*
`v2–v3`	Heavy-hitters domain; Pareto-frontier reward; the Adversary attacks the *metric* and hardens the suite; a compositional grammar that re-derives Space-Saving.	Genuine compositional invention, but within a hand-designed basis. *Open the vocabulary itself.*
`v4–v5`	Policies composed from low-level ops; Builder and Adversary author *new ops and generators as source code*, screened and grounded before admission; provenance tracking.	The machine can author and ground a primitive outside its vocabulary — but it only *matches* the best human. *Proposal quality is the new ceiling.*
`v6`	Causal guidance (feed the proposer the actual failing trace); self-play; explicit beats/matches/below-human utility flags.	First machine primitive to *beat* the best human (`declow` , brute-force-confirmed) — but only on the memory axis, on a recall-saturated domain. *Change the problem.*
`v7`	Cache-eviction domain with a Bélády oracle and real headroom.	Reported **3 beats-human** on the primary metric — but measured on a run-variable suite with a noisy cost axis. *Make the metric unfakeable and reproducible.*
`v8`	The trusted driver (§4); exact-capacity cost; a fixed evaluation suite; the propose→compose loop closed.	Applying that rigor to v7's own claims, the honest reproducible count fell to **1**. The deliverable was *trust*, not a bigger number. *No static score dominates all regimes — need adaptive policies.*
`v9`	Bounded adaptive state in the trusted driver (regime detection); competitive-ratio quality; a declared weighted benchmark.	First *robust* wins and the first **open-invented** beats-human from capable models — but the human baseline was still only the four static classics. *Raise the bar to the humans' adaptive algorithms.*
`v10`	ARC and a LeCaR-style policy as adaptive human baselines, measured through the identical harness; two-tier grading; a completeness gate; enforced statelessness.	Surprisingly, the adaptive wins *held* against ARC/LeCaR. But the bar was still three capacities, one seed, two reference algorithms. *Validate, broaden, and stress the bar until it cannot flatter a candidate.*
`v11`	A design constitution; dual-validated references; 2Q + W-TinyLFU added; multi-seed measurement; a cost-charge sweep; a fresh hold-out family; rubric-scrubbed prompts.	**The streak ends:** 0/10 prior winners survive the broadened bar; the two that still beat on the home suite are flagged overfit by the hold-out. *The real frontier work is generalization, not benchmark margin.*

Three times the project reported a worse number than the version before — on purpose, and counted each as a success. That is the behavior a trustworthy evaluation must have and almost none do.

The v7→v8 collapse, in detail

It is worth dwelling on the moment the discipline first bit. Version 7's "three beats-human" was not fraud; it was the ordinary way benchmark numbers inflate. The cost axis was a noisy memory measurement, so a proposed op could look marginally cheaper than an equally-capable human policy and thereby "win" a comparison it had not earned; and the evaluation suite varied from run to run, so the win was partly an artifact of which attacks happened to be admitted that run. Version 8 made the cost an exact integer and fixed the suite. The same ops were re-graded. Two of the three wins dissolved. The project wrote that down as its headline result and moved on. A research system that exposes its own inflation is doing precisely the thing the field most needs evaluation to do.

06 v11: the design constitution and the end of the streak

By version 11 the project had learned a deeper lesson than any single fix. It is not enough to make one metric unfakeable. You need a *policy* for every metric you will ever add — a way to decide, in advance, which numbers may touch the optimization loop at all. That policy is the design constitution.

Three kinds of metric

Table 2 — The v11 design constitution. Every metric is classified before it is wired in.

CLASS	DEFINITION	WHERE IT IS ALLOWED TO LIVE
Unfakeable	Computed by trusted code that ran the candidate (e.g. the competitive ratio).	In the loop, and in the machine-checkable claims file.
Alignable	A metric where *gaming it is the goal* — e.g. behavioral distance, where being genuinely different is the point.	Loop-visible. There is no exploit, because the exploit is the objective.
Insulatable	A judgment *about the generator*: "was this invented or interpolated?", "is the write-up honest?"	**Never computed in the repo.** The repo emits an evidence bundle; an external reviewer judges; findings return only as problems.

The third row is the most striking decision in the project. The repository **stopped trying to answer "was this invented?"** Any in-house metric for inventiveness would itself become a target the generator could optimize toward, manufacturing the *appearance* of novelty. So that judgment was moved entirely outside the system. Each run emits an audit bundle — the post-scrub prompts, every proposed fragment including the rejected ones, a template-library hash stamped against the benchmark version — for an external reviewer to assess. The only novelty signal kept *inside* the loop is behavioral distance, which is alignable: a candidate that scores well on it is, by definition, behaving differently, which is exactly what was wanted.

> **THE LEAK RULE**
>
> Information may flow from evaluation back to generation — but only of one kind. It may carry **problems**: the trace a policy failed on, the keys it missed, the verbatim text of a contract error. It may *never* carry **rubrics**: the epsilon, the benchmark weights, the worked idioms, the thresholds. Telling a generator *what went wrong* sharpens it; telling it *how it will be scored* teaches it to overfit the grader. The rule draws that line precisely, and a test enforces it.

The rubric scrub

The leak rule was applied with unusual severity. The proposer-facing prompts had, over earlier versions, accumulated worked examples — a canonical adaptive-state update written out, a threshold-switch policy with its exact constants, sentences explaining what scores well. All of it was removed. The reason is forward-looking: a

capable-model run on the *scrubbed* prompts would be the first clean reading of what the models produce given **capability without idiom** — invention rather than imitation of a taught pattern. Tellingly, the project treats "the models produce nothing admissible without the worked example" as a live and *informative* possible outcome, not a failure to be engineered around.

Making the bar real — and the streak ending

In parallel, the bar itself was hardened along four axes, then frozen (the freeze is visible in the git history):

- **Validated references.** An independent ARC implementation was written from the original paper's case structure, using different data structures, and required to agree *hit-for-hit* with the canonical one across all 63 trace×capacity combinations. It did.
- **Broadened tier.** 2Q and W-TinyLFU — among the strongest known policies — joined ARC and LeCaR, all charged for their metadata through a single cost-accounting site.
- **Multi-seed measurement.** Five seeds per workload, with a declared rule for when a margin counts as a win versus when it is "fragile" (straddling the boundary across seeds), and a selection penalty that scales the bar with the number of candidates graded.
- **A charge sweep and a hold-out family.** The cost of metadata was swept across four settings; a win counts as *robust* only if it survives all of them. And a fresh family of workloads — same regimes, freshly drawn parameters and seeds — was generated at the end to test whether wins transfer.

Table 3 — The frozen-bar re-grade of every prior winner. "Declared charge" is the single nominal cost setting; "robust" means winning at *every* point of the charge sweep.

RE-GRADED POPULATION	BEATS AT DECLARED CHARGE	ROBUST BEATS (FULL SWEEP)
10 prior v9/v10 winners	**7 of 10**	**0 of 10**
14 ops in the citable v11 run	2 of 14	**0 of 14**

The mechanism of the collapse is precise. When metadata is charged nothing, W-TinyLFU (competitive ratio ~0.93) and 2Q at resident-only cost dominate every candidate the project had produced. Version 10's claim that its wins "survive ghost-free accounting" had been true only against ARC and LeCaR; the stronger references close the gap. Every op that still beat at the nominal charge was adaptive; every static blend fell to a fragile match — exactly the knife-edge the multi-seed rule was built to expose.

THE HOLD-OUT FINDING — THE RUN'S REAL HEADLINE

The two ops that still beat on the home benchmark were sent against the freshly-drawn family. Both lost decisively to W-TinyLFU:

```
ad_thrash   bench +0.035  →  hold-out -0.171
ad_guard15   bench +0.059  →  hold-out -0.137
```

A swing of roughly 0.2 against a 0.05 overfit threshold. The threshold-switch idiom's wins were calibrated to the declared suite's exact parameters; on cousins of the same regimes they did not transfer. Both ops carry `overfit: true` in the run's claims file. The verdict layer now says, with evidence at every level: **the current family of policies does not robustly beat the best adaptive humans, and the real work ahead is generalization, not benchmark margin.**

One quiet corroboration is worth noting. The behavioral-distance signal — the only in-loop novelty metric — independently flagged those same two ops as the most behaviorally distant from any reference. The inside-the-loop signal and the outside hold-out judgment agreed, without the repository ever scoring "inventiveness" directly.

07 What this teaches about the frontier of AI

RDE is a small system on humble problems. But it was built around exactly the questions that dominate the frontier, and the discipline it enforces generalizes cleanly. Seven lessons are worth carrying forward.

1 — GENERATION IS CHEAP; TRUSTWORTHY VERIFICATION IS THE BOTTLENECK.

Almost the entire system is verification machinery wrapped around a tiny generator. As models get more capable, this ratio is the future. The scarce, valuable engineering is not the part that proposes — it is the part that can be trusted to say whether the proposal is real.

2 — THE GENERATOR–GRADER SEPARATION MUST BE STRUCTURAL, NOT PROCEDURAL.

A rule that says "the model shouldn't grade itself" is a procedure, and capable systems route around procedures. The trusted driver makes the separation *structural*: the grade is computed by code the candidate cannot reach. Wherever you can run an artifact rather than read its self-report, you can build this guarantee.

3 — REWARD HACKING IS THE DEFAULT, AND EVERY NEW DEGREE OF FREEDOM OPENS A NEW ONE.

Each capability added to the candidate contract opened an exploit, and an adversarial review *reproduced* each as a working attack before it was closed: a free-form score that could report the optimum; a capacity knob that could be charged cheap while run expensive; rebinding tricks that slipped past a naive check. Each was closed structurally — an exact literal-only knob, a whole-program binding census, a single cost-accounting site. The lesson is not "patch the exploits" but "expect every new freedom to be one, and design the closure into the contract."

4 — MOVE JUDGMENTS THE MODEL COULD RATIONALIZE OUT OF THE MODEL.

"Are we making progress?" is computed from history and handed to the Synthesizer as a fact, not asked of it. "Was this invented?" is removed from the repository entirely and given to an external reviewer. The principle: any judgment a capable system has an incentive to shade is a judgment that system should not be the one to make.

5 — DISTINGUISH PROBLEMS FROM RUBRICS IN EVERY FEEDBACK CHANNEL.

The leak rule is a precise, implementable statement of how to give a generator feedback without teaching it the test. Failing traces, missed cases, verbatim errors — yes. Weights, thresholds, worked idioms — no. This single distinction separates feedback that builds capability from feedback that manufactures benchmark-fit.

6 — OVERFITTING HIDES INSIDE BENCHMARKS; HOLD-OUTS DRAWN FROM THE SAME FAMILY FLUSH IT OUT.

A win on a fixed suite is consistent with both real capability and quiet memorization of the suite's quirks. The decisive test is a freshly-drawn family of the same kind of problem. The two v11 ops that looked like wins were unmasked the moment they faced cousins they had not been tuned to. For anyone reading model evaluations, this is the question to ask of every reported number: *does it transfer to a draw the system never saw?*

7 — HONEST CULTURE IS LOAD-BEARING, NOT DECORATION.

The architecture only produces trust because “the streak ended” is reported plainly and a worse number is counted as a success. A system that hides its own inflation is worse than no system, because it spends credibility it has not earned. The rule “never tune the gates, weights, seeds, or epsilon to preserve a headline” is what makes the final null result believable — and it is exactly the rule the broader field most often breaks.

> **THE UNCOMFORTABLE CONCLUSION, STATED CLEANLY**
>
> After twelve generations of increasingly capable proposers — including frontier models proposing freely — the honest finding on this domain is that *nothing robustly out-performs the best human algorithms once you measure carefully, charge fairly, and test on fresh draws.* That is not a failure of the engine. It is the engine working: it is what an honest discovery system is *supposed* to tell you when the genuine discovery has not yet happened. The value is in being able to believe it.

08 Limits, open questions, and what comes next

In the spirit of the project, here is what RDE is *not*, and what remains genuinely open.

What it is not

- **Not autonomous science.** RDE is a disciplined harness on small algorithmic domains with known oracles. It is a model of the verification problem, not a system doing open-ended research. Its honesty about scope is part of its value.
- **Not a hardened sandbox.** The code-execution defenses are layered and adequate for a trusted local setting; a determined adversary with an unrestricted standard library could find gaps. The report says so directly rather than implying more safety than it has.
- **Not de-novo invention, yet.** The wins it has produced were authored within idioms the proposer had been shown. Whether capability without those idioms invents anything admissible is precisely the open question below.

The pending experiment: capability without idiom

The single most interesting open item is a capable-model run on the rubric-scrubbed prompts — the first clean reading of what frontier models produce when the worked examples have been removed and only the problem remains. All four outcomes are publishable: a robust win, a fragile win, a benchmark-only win, or nothing admissible at all. That last outcome — "the models invent nothing the bar will accept once the idiom is gone" — would itself be a significant and honest finding about the current frontier. The run is deliberately gated on operator confirmation and a credential rotation; it has not been launched.

Where the frontier work actually is now

- **Generalization, not margin.** The hold-out result relocated the goalpost. Candidate mechanisms must win on *families* of workloads, not points on a fixed suite. An objective that rewards transfer directly is the natural next design.
- **A still-stronger reference tier.** The adaptive bar lacks LIRS and uses one deterministic LeCaR variant. Every reference added has, so far, made the project's own claims harder to sustain — which is the point.
- **A second trusted-driver domain.** Admission control or bin-packing, built on the same generic gate, would show that "trusted driver + competitive ratio + declared benchmark" is an architecture rather than a cache-specific trick.

A NOTE ON REPRODUCIBILITY

The verdict layer is deterministic: ten independent runs sharing an admitted vocabulary produce bit-identical grades. What is load-sensitive is vocabulary *formation* — under heavy machine load the proposal trajectory shifts, so a different set of ops gets proposed. The claims file makes this detectable rather than silent by recording how many ops were graded, and the bar scales with that count. Even the system's own non-determinism is surfaced as data instead of hidden.

09 Closing: the discipline of the disappointing result

It would have been easy to ship this project as a success story. Several versions produced real, defensible "machine beats human" headlines, and each was true under the bar of its day. The choice that makes RDE worth a frontier audience's attention is that it kept raising the bar *after* clearing it — and reported, without flinching, when the headlines fell.

The deepest result is therefore not in the cache domain at all. It is a demonstration that you can build an AI system whose success metric is robust to the intelligence operating inside it: where the grade is computed, not reported; where the judge that may declare victory is fed facts it cannot rationalize away; where the question "did it really invent something" is held outside the system because the system would otherwise learn to fake the answer; and where the operating culture treats a worse, truer number as the goal. When such a system is run honestly across twelve generations and frontier proposers, and its verdict is "not yet," that verdict is worth more than any headline it discarded along the way.

That is the frontier this project actually advances. Not "the machine discovered a better cache policy." The harder and more valuable thing: **a machine you can believe when it tells you it hasn't.**

> **A result that lowers the headline is a success.**
>
> *— the rule the whole system is built to obey, and the one the field most needs to learn.*

About this document. This is a narrative field report on the Recursive Discovery Engine, written for readers tracking the frontier of AI. Every quantitative claim in it traces to the project's own machine-checkable artifacts: the versioned README, the v11 design notes, the frozen-bar re-grade script, and the per-run claims files, each of which is reproducible from the codebase. Numbers are reported as the project reports them — including, deliberately, the ones that fell.

Recursive Discovery Engine · Python standard-library core · OpenRouter LLM transport · domains: top-K selection, streaming heavy-hitters, cache eviction · 169 tests · v0–v11.